Open Source Formal Verification

Introduction to hardware verification

Yann Thoma

Reconfigurable and Embedded Digital Systems Institute
Haute Ecole d'Ingénierie et de Gestion du Canton de Vaud



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License

November 2026

1. Introduction
2. Simulation-based verification
3. Formal verification
4. Automatic verification
5. Simple testbenches
6. Emulation-based verification
7. Languages
8. Trends

Verification goal

Answer two questions

- Does it work?
- Are you sure?
- Really?
- Well, really really sure?
- No kidding?

Verification : yes, but ...

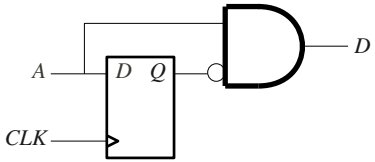
We verify ...

- What?
  - FPGA design
  - Software
  - Both
- Why?
- When?
- How?

First question: What?

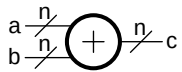
- Verification of the correct behavior of the system
- What does that mean?
- Verification of each and every module
  - Is exhaustive verification affordable?

- An edge detector:



  - Exhaustive verification: OK

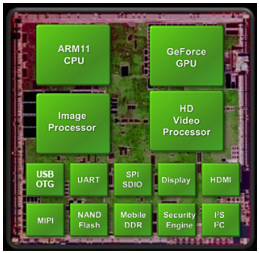
- A 64-bits adder:



  - Exhaustive verification: No
  - $2^{128} = 3.4 \times 10^{38}$ combinations...

First question: What?

- Verification of the correct behavior of the system
- What does that mean?
- Verification of the whole system (System on Chip for instance)
  - Exhaustive verification unfeasible
  - $\Rightarrow$ Choose interesting cases
  - $\Rightarrow$ Random verification



http://www.xbitlabs.com/news/mobile/display/20080603141353_Nvidia_Unleashes_Tegra_System_on_Chip_for_Handheld_Devices.html

Question: Why?

- Patching an FPGA remotely is not always ideal
- Also, it can harm the system (you know that)
- Verification required to validate the system
  - Needs efficient tools
  - A rigorous methodology
- Verification is *the challenge* of digital design

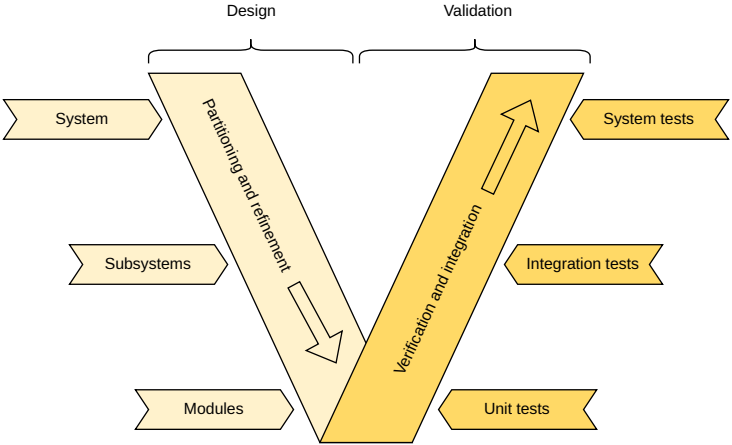
Verification: concept

- Goals
  1. Assess the conformity of the system relative to the specifications
  2. Detect errors as soon as possible
  3. Ensure a correct behavior after synthesis and integration
- Difficulties
  1. To keep the verification cost low
     - Around 50-60% of time spent on verification
  2. To guarantee a correct behavior

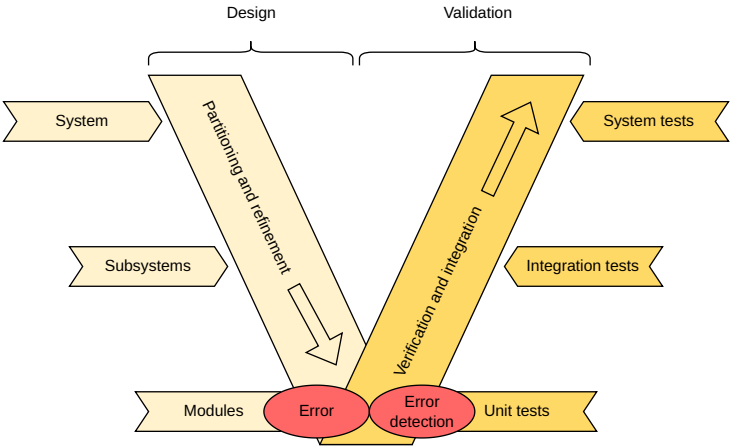
Question: When?

- Obviously as soon as possible
- Not after hours spent trying to spot why the system does not work
- Unit tests on all modules during design and implementation
- System tests or integration tests as soon as possible
  - Harder to setup
  - Huge simulation times
  - When? At night...

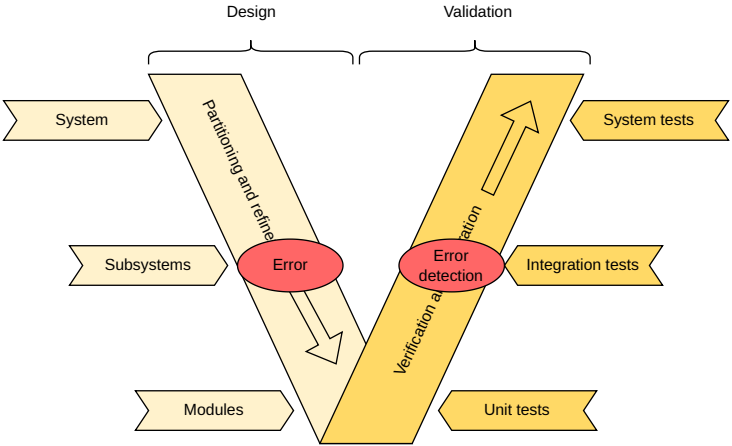
Design/Verification: decomposition



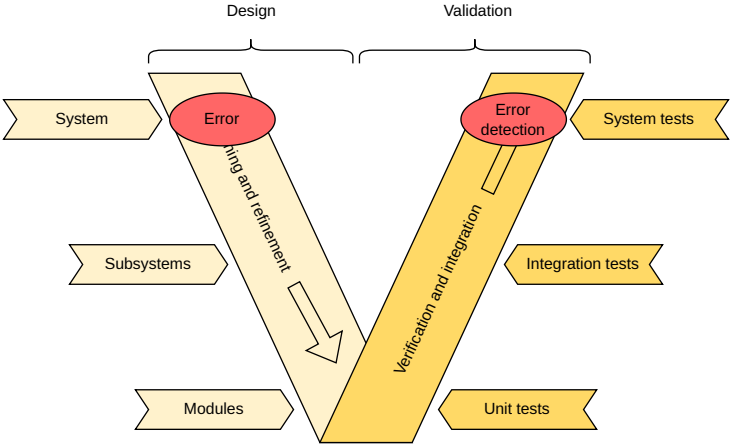
Design/Verification: decomposition



Design/Verification: decomposition



Design/Verification: decomposition



Verification: the 2 questions

- Two big questions during verification
  1. Does it work?
     - Does the system behave as expected?
     - Are errors correctly detected?

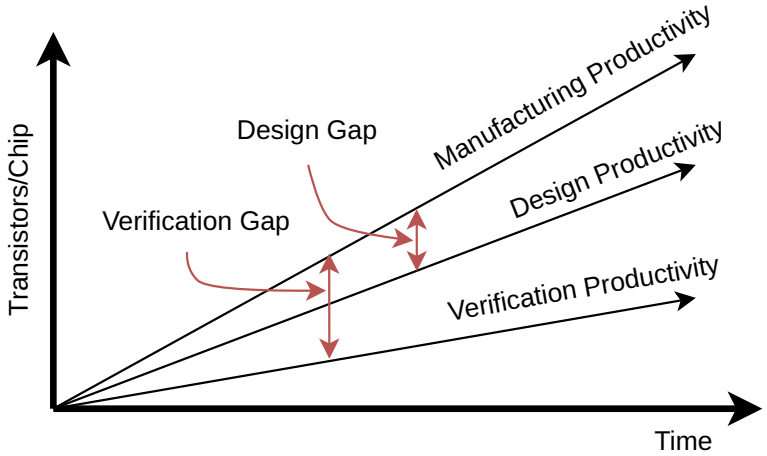
Example of a testbench that passes all tests

```
entity my_great_design_tb is
end my_great_design_tb;

architecture my_great_architecture of my_great_design_tb is
begin
    process is
    begin
        report "Everything is fine";
    end process;
end my_great_architecture;
```

  2. Are we sure? (Are we over?)
     - Did we run enough tests?
     - Easy with a small one, quite difficult with a complex design

Verification: Evolution of Complexity



Question: How?

- Language choice
  - VHDL
  - SystemVerilog
  - SystemC
  - Python
  - ...
- Methodology choice
  - UVM
  - OSVMM
  - UVVM
  - ...
- Technological choice
  - Formal verification
  - Simulation-based verification
  - Emulation
- Interactive
  - Manual or automatic

Challenges of verification: methodology choice

- The methodology choice should take into account:
  - Completeness
    - Maximizing the number functionalities (scenarios) tested
  - Reusability
    - Maximizing code reusability for future projects
  - Efficiency
    - Minimizing the effort, and maximizing automation
  - Productivity
    - Maximizing useful verification progress per unit of engineering effort
    - Automate repetitive tasks; focus manual effort on specification review, property design, and failure analysis
  - Code performance
    - Minimizing computing time

Functional verification: approaches

- Simulation-based verification
  - Executes selected testbench scenarios in a simulator
  - Finds bugs on exercised traces; coverage shows what was exercised, not all possible behavior
- Emulation-based verification
  - Runs test workloads on FPGA-based emulation hardware, often accelerating system-level scenarios
  - Like simulation, it explores behavior under applied stimuli
- Formal verification
  - Checks encoded properties under the model and assumptions
  - A bounded check covers only its configured depth; an unbounded proof covers all modeled behaviors

Simulation-based verification: approaches

- Different approaches to simulation-based verification:
  - Directed tests
    - Manually-designed tests
    - For instance: exhaustive tests (!)
  - Coverage-driven random-based
    - Generation of random stimuli
    - Coverage-driven for determining the end of the simulation
  - Assertion-based verification
    - Assertions can be used for formal verification
    - But also used in simulation
- Tools (languages) are more or less suited for one or the other approach

Combining approaches: FIFO example

- **Directed simulation:** Test reset, fill, drain, and read/write boundary cases according to the FIFO interface contract.
- **Constrained-random simulation:** Generate legal read/write traffic and track occupancy and boundary coverage.
- **Assertions and scoreboard:** Check protocol rules and data ordering on each transaction; reuse assertions in formal where supported.
- **Formal verification:** Under explicit interface assumptions, prove selected safety properties and cover boundary scenarios. A bounded check is limited to its configured depth.

Use them together (or not)

Simulation explores selected scenarios; formal checks properties across the behaviors admitted by its model and assumptions.

Objectives of simulation

- Exercise selected directed and randomized input scenarios
- Check expected behavior with self-checking testbenches, scoreboards, or assertions
- Measure cycle-level behavior, such as latency and throughput
- Produce pass/fail results and logs for CI; record test configuration and random seed for reproducibility
- Coverage reports show which scenarios were exercised, not that all behavior is correct
- RTL simulation does not establish maximum clock frequency; use static timing analysis for timing closure

Verification methodology

- Exhaustive verification
  - The number of cases can grow exponentially with input width
  - Two 64-bit inputs have $2^{128} \approx 3.4 \times 10^{38}$ possible pairs
  - Sequential designs add state and input histories, increasing the search space
  - Formal tools can reason symbolically, but verification can still be computationally challenging
- *Standard* verification
  - Test of the code (white box)
    - Code coverage
  - Test of the behavior (black box)
    - Verification of the system behavior, without any information on its internal architecture
    - Ideally should be done by another person than the design developer
- Advanced approaches
  - Assertions, random-based coverage-driven, ...

Verification methodology

- We should avoid testbenches that do not detect design flaws
  - Very easy to write a happy testbench that doesn't detect any errors...
  - Rigor is essential in writing a testbench
  - Testbench auto-test is a good idea
    - The testbench modifies the design outputs to check itself

Simulation output validation

- For direct verification, validation is given by:
  - List of the generated stimuli et verifications done (testbench files)
  - Written proof of the verification results (simulator log, text file, HTML, junit XML, ...)
- For random-based verification, validation is given by:
  - List of the tested scenarios et verifications done (testbench files)
  - Coverage
  - Written proof of the verification results (simulator log, text file, HTML, junit XML, ...)

Formal verification

- Formal verification checks a model against encoded properties
- Results are conditional on the model, properties, and assumptions
  - A completed proof establishes the properties for all modeled behaviors under the assumptions
  - Bad properties $\Rightarrow$ false results
- A counterexample provides a trace that violates a property
- A proof does not show that the specification is complete, assumptions are justified, or implementation and integration match the model
- Formal verification usually complements simulation and system-level testing

Where is formal verification a good fit?

Often a strong fit

- Control-dominated blocks: FSMs, arbiters, counters, protocol controllers
- Safety properties: mutual exclusion, legal state transitions, no FIFO overflow/underflow
- Corner cases that are difficult to reach with simulation
- Small blocks with clear interfaces and assumptions

More challenging on its own

- Very wide datapaths or large arithmetic structures at bit level
- Large systems checked monolithically, without abstraction or decomposition
- Analog, physical, or timing behavior absent from the model
- Properties that exceed the chosen tool flow's supported features

Suitability depends on the property, model, and tools. Abstraction or decomposition can help; formal verification complements simulation.

Manual vs. automatic

- Manual verification
  - Forcing input signals *by hand*
  - Huge risk of errors
  - Hard to to execute the same scenario twice
  - Visual verification by the developer, thanks to simulation chronograms
  - Slow and error-prone
  - Risk of bad analysis by the developer
- semi-automatic verification
  - Input signals applied automatically
  - Visual verification by the developer, thanks to simulation chronograms
  - Risk of bad analysis by the developer

Automatic verification

- Goal: verification as complete as possible
- Results in a Go/No Go (OK, KO)
- Indicates the time of error
- Enables a complete rerun (Go/No Go) after a modification or correction
- Perfect for continuous integration
- Interactions with humans should be minimized
  - Required to ensure that reruns are similar

Automatic simulation: scripts

- Obviously, scripts are used to start simulations
- Avoid human errors
- Can be shared by members of the team
- Make life much simpler

Example: sim.do for QuestaSim

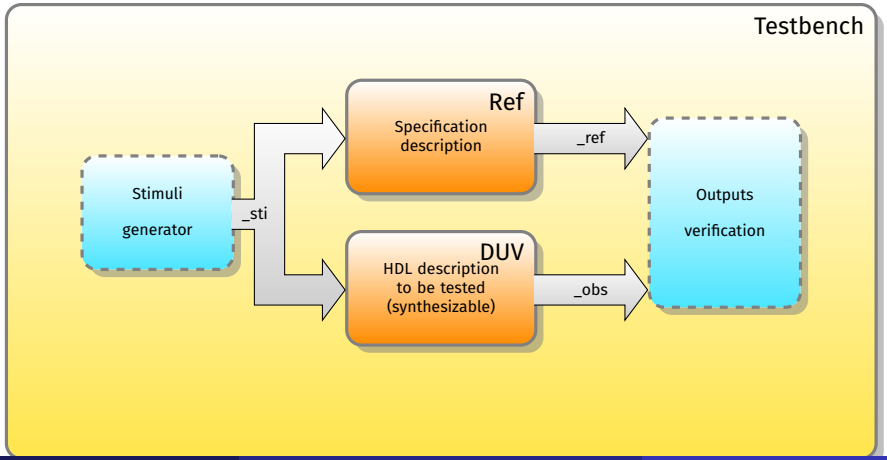
```
# Definition of the work library
vlib work
# compilation of the DUV
vcom counter.vhd
# compilation of the testbench
vcom counter_tb.vhd
# launching of the simulation
vsim work.counter_tb
# Addition of all signals to the chronogram
add wave -r *
# Simulation run
run -all
```

Black/white/gray box verification

- Goal of a design: the outputs respect the specifications
- But... Some internal errors could be untrackable
  - Never-activated errors
  - Errors that do not propagate to the outputs
  - Errors canceling each other
- Black box
  - Only the design outputs are available for verification
  - Typically compared to a reference model

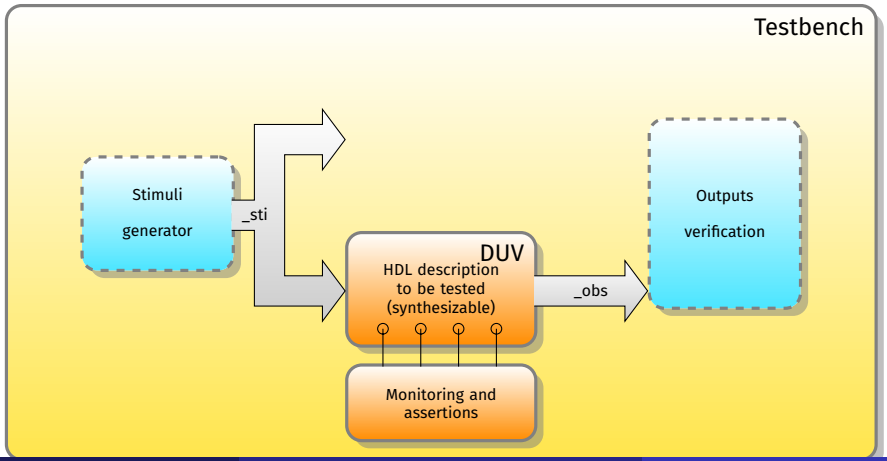
Black box

- Only testing the outputs
  - Hard to find the source of errors
  - Requires a reference model



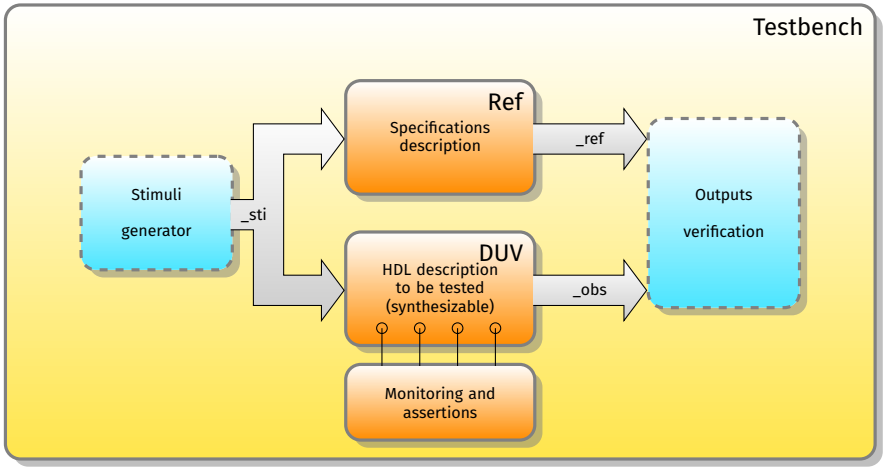
White box

- The module architecture is analyzed
  - Easier to look for errors
  - Can be limited for complex systems



Grey box

- The output and internal architecture can be analyzed
  - Compromise between both approaches



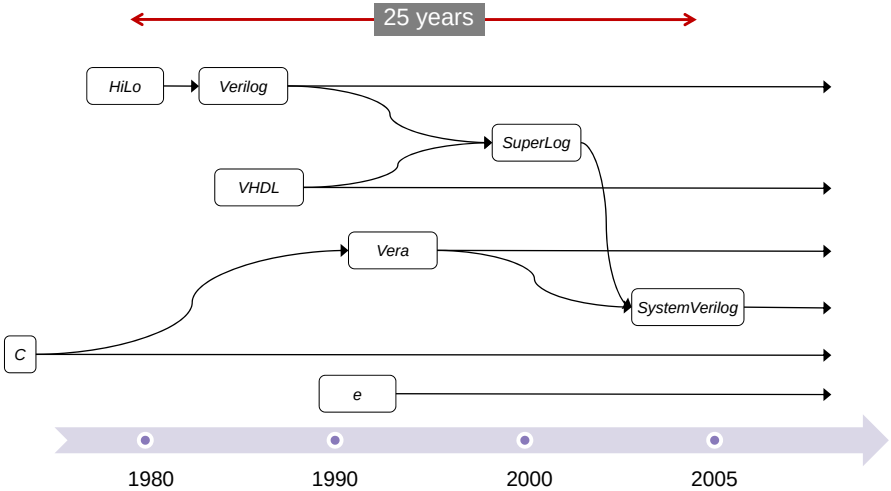
Emulation-based verification

- Issue with simulation: execution time can be huge
- Solution: Exploit FPGAs to accelerate the process
- Difficulties:
  - Placement and route computing time
  - Observability (number and time of windows)
  - Assertions
  - Quite well for black box verification
- Commercial solutions:
  - Veloce (Siemens)
  - ZeBu (Synopsis)
  - Palladium (Cadence)

Languages for verification

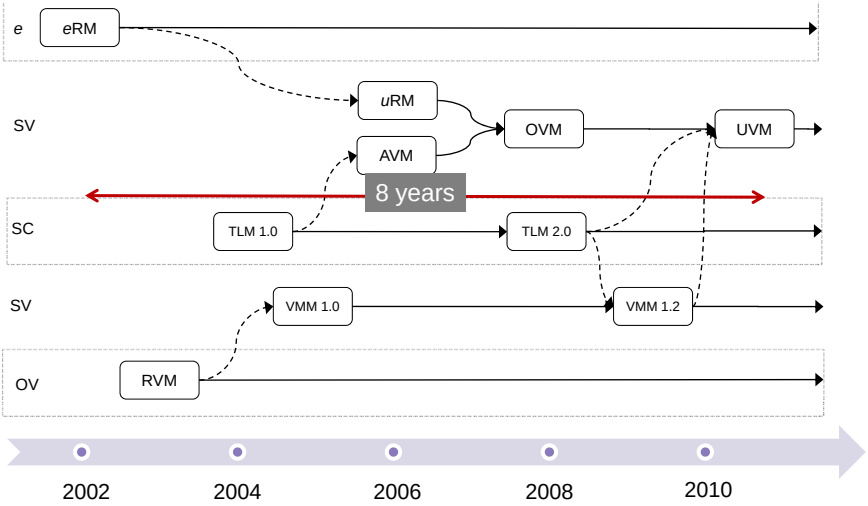
- Small to middle-size projects
  - VHDL
- Middle to high-size projects
  - Needs for:
    - High-level models (transaction)
    - Random stimuli generation
    - Assertion-based verification
  - Current options
    - SystemVerilog
    - SystemC
    - Python
    - Assertions: PSL (*Property Specification Language*), or SVA

Origin of languages



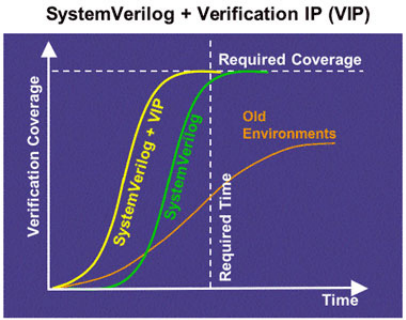
Functional Verification of Today's and Tomorrow's SoCs, Janick Bergeron, Synopsys, 2011

Origin of methodologies



Functional Verification of Today's and Tomorrow's SoCs, Janick Bergeron, Synopsys, 2011

SystemVerilog vs. VHDL



Source: http://www.design-reuse.com/articles/7844/verification-ip-verification-ip--part-2-by-dr-aart-de-geus.html

SystemVerilog

- Object-oriented
  - Inheritance
  - Dynamic creation of objects
- Interfaces
- Reusability
- Random generation
  - Unconstrained
  - Constrained
- Assertions
- Coverage
- Ideal for random-based coverage-driven verification
- A set of methodologies

SystemVerilog Methodologies

- Synopsys: VMM (Verification Methodology Manual)
  - Version 1.2
  - `https://www.synopsys.com/verification/vmm-central.html`
- Mentor: AVM (Advanced Verification Methodology)
  - Version 3.0
  - Terminated
- Ensemble: OVM (Open Verification Methodology)
  - Version 2.1.2 $\rightarrow$ UVM
  - `http://www.ovmworld.org/`
- UVM (Universal Verification Methodology)
  - Unification de VMM et OVM
  - Accellera standard!
  - Version 2017-1.0
  - `https://www.accellera.org/downloads/standards/uvm`

Abstraction levels



Abstraction levels



TLM methodology

- For complex systems, the *Transaction Level Modeling* offers:
  - Better modularity
  - Better reusability
  - Decoupling of RTL against other abstraction levels
- Concept:
  - Go to higher abstraction levels as possible
  - Be as much as possible in a software world
- SystemVerilog is well-suited
- VHDL approaches that with UVVM and OSVVM
- If formal is too complex, then switch to TLM

Cocotb: python-based verification

- Cocotb is a python-based verification framework
- It allows for the development of testbenches in Python
- It can be used with both VHDL and Verilog
- It provides a high-level interface for interacting with the simulation
- Can be used on real hardware tests also

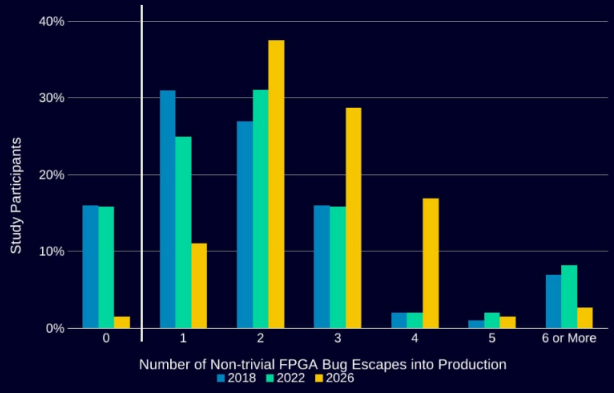
Study on worldwide usages for design and verification

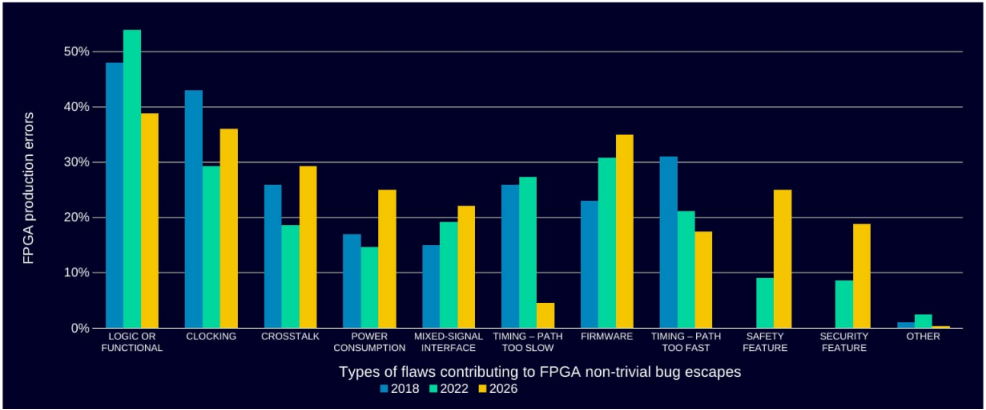
- *2026 Wilson Research Group FPGA functional verification trend report*
- Published by Siemens
- Comparison between 2018 et 2026 (started in 2012)
- Designs FPGA and ASIC (but let's focus on FPGA)

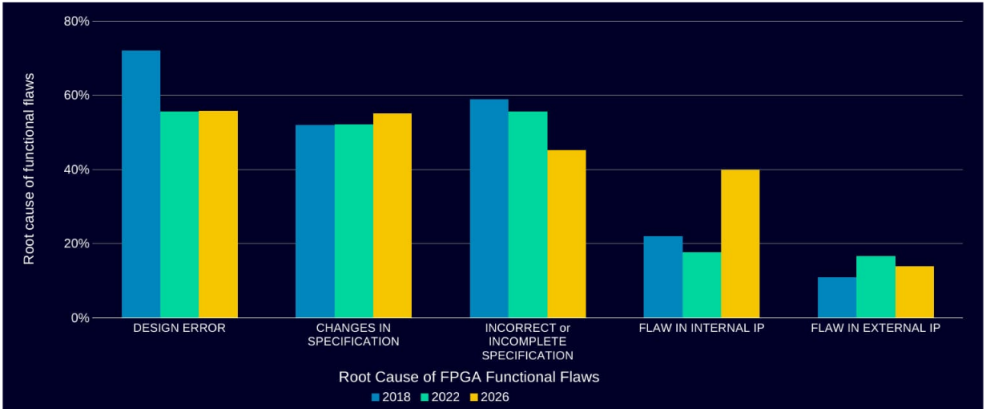
https://verificationacademy.com/topics/planning-measurement-and-analysis/wrg-industry-data-and-trends/2026-functional-verification-data-atlas-reference/

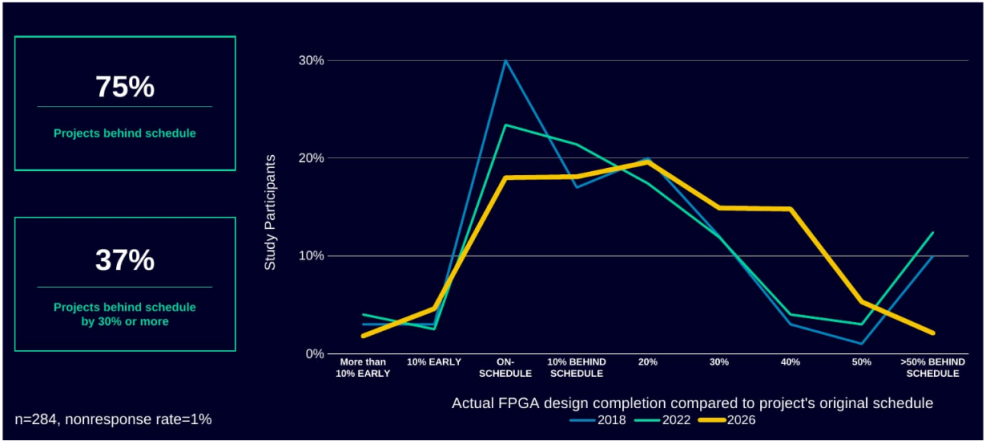
98%

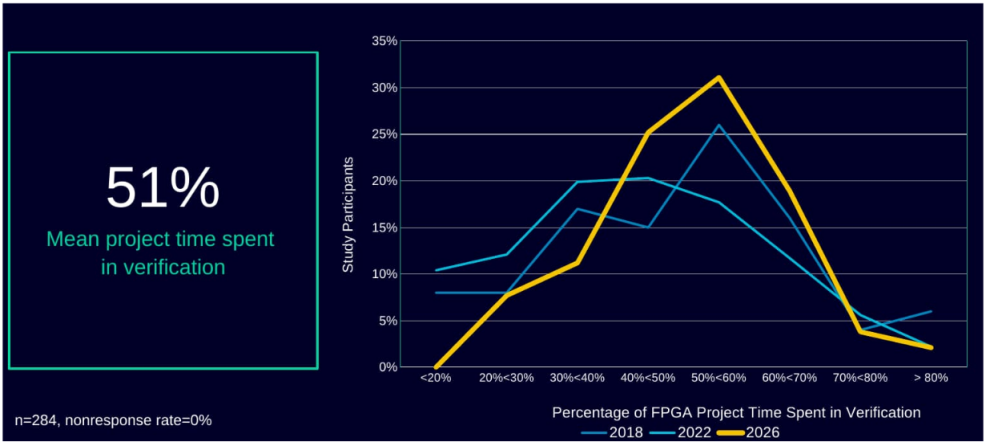
FPGA Study Participants have non-trivial bug escapes into production

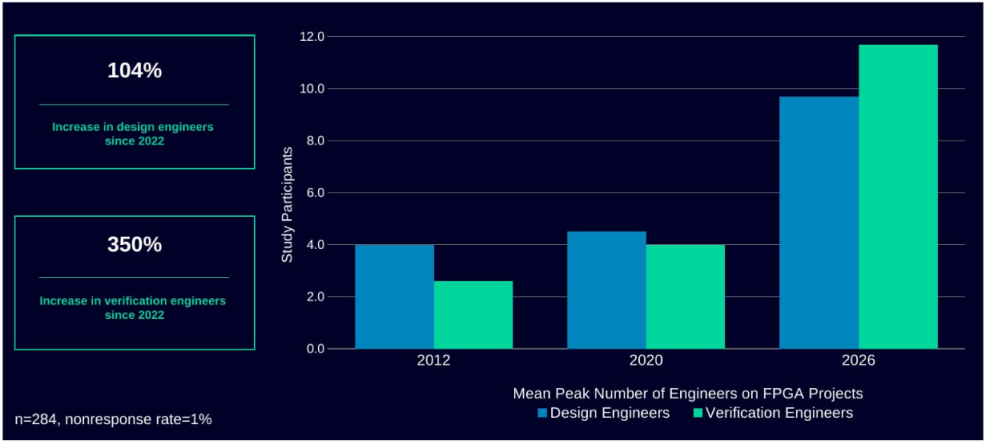


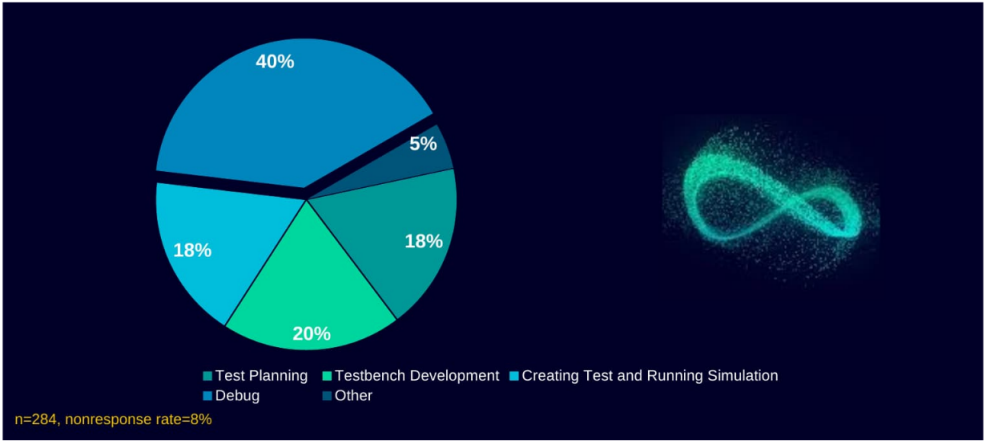

FPGA production errors
50%
40%
30%
20%
10%
0%
LOGIC OR FUNCTIONAL
CLOCKING
CROSSTALK
POWER CONSUMPTION
MIXED-SIGNAL INTERFACE
TIMING – PATH TOO SLOW
FIRMWARE
TIMING – PATH TOO FAST
SAFETY FEATURE
SECURITY FEATURE
OTHER
Types of flaws contributing to FPGA non-trivial bug escapes
2018
2022
2026

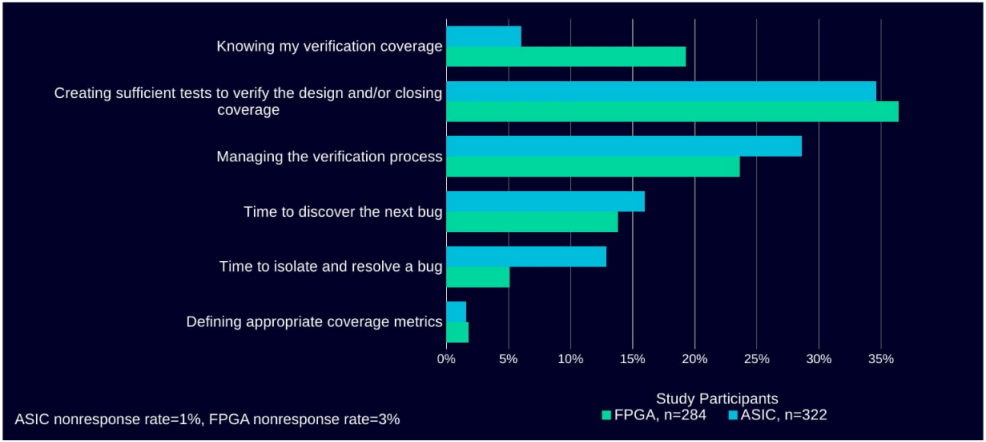

80%
60%
40%
20%
0%
Root cause of functional flaws
DESIGN ERROR
CHANGES IN SPECIFICATION
INCORRECT or INCOMPLETE SPECIFICATION
FLAW IN INTERNAL IP
FLAW IN EXTERNAL IP
Root Cause of FPGA Functional Flaws
2018
2022
2026

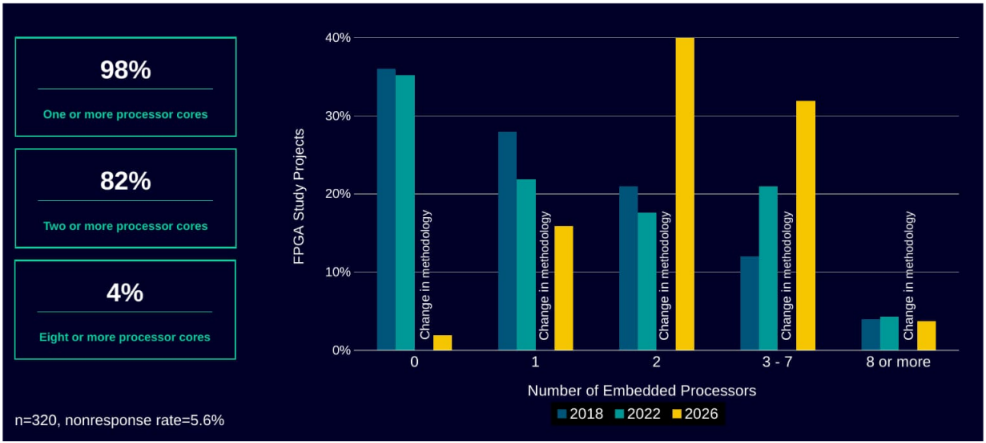

75%
Projects behind schedule
37%
Projects behind schedule by 30% or more
30%
20%
10%
0%
Study Participants
More than 10% EARLY
10% EARLY
ON-SCHEDULE
10% BEHIND SCHEDULE
20%
30%
40%
50%
>50% BEHIND SCHEDULE
Actual FPGA design completion compared to project's original schedule
2018
2022
2026
n=284, nonresponse rate=1%

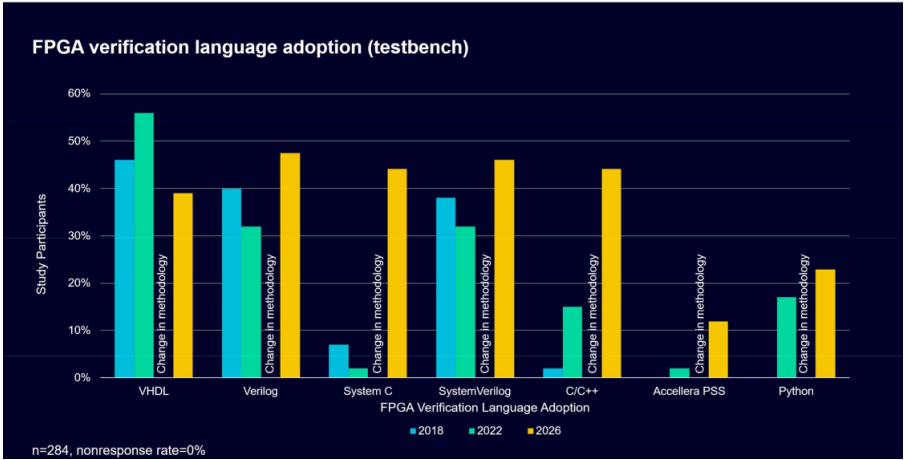

51%
Mean project time spent in verification
n=284, nonresponse rate=0%
Study Participants
35%
30%
25%
20%
15%
10%
5%
0%
<20%
20%<30%
30%<40%
40%<50%
50%<60%
60%<70%
70%<80%
> 80%
Percentage of FPGA Project Time Spent in Verification
2018
2022
2026

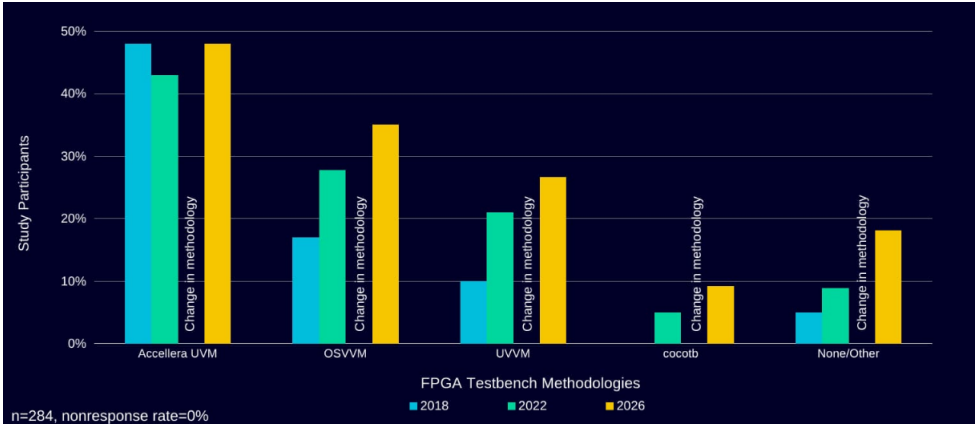

104%
Increase in design engineers since 2022
350%
Increase in verification engineers since 2022
12.0
10.0
8.0
6.0
4.0
2.0
0.0
Study Participants
2012
2020
2026
Mean Peak Number of Engineers on FPGA Projects
Design Engineers
Verification Engineers
n=284, nonresponse rate=1%

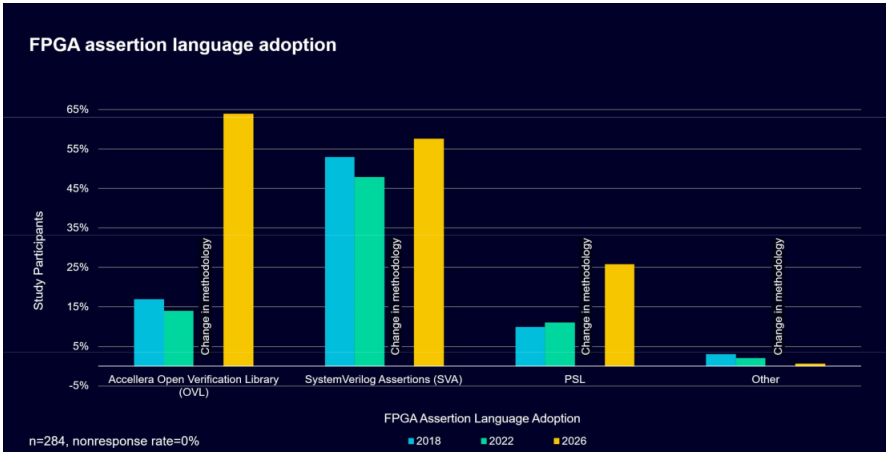

40%
5%
18%
18%
20%
Test Planning
Testbench Development
Creating Test and Running Simulation
Debug
Other
n=284, nonresponse rate=8%

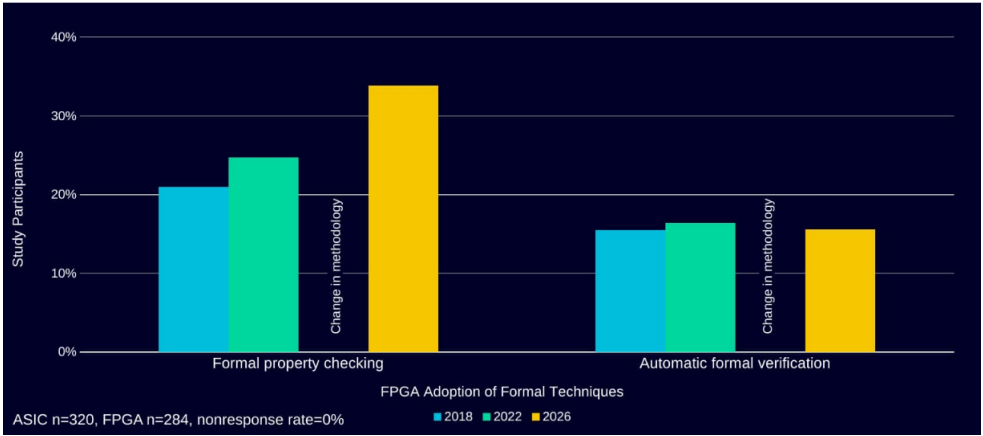

Knowing my verification coverage
Creating sufficient tests to verify the design and/or closing coverage
Managing the verification process
Time to discover the next bug
Time to isolate and resolve a bug
Defining appropriate coverage metrics
0%
5%
10%
15%
20%
25%
30%
35%
Study Participants
FPGA, n=284
ASIC, n=322
ASIC nonresponse rate=1%, FPGA nonresponse rate=3%

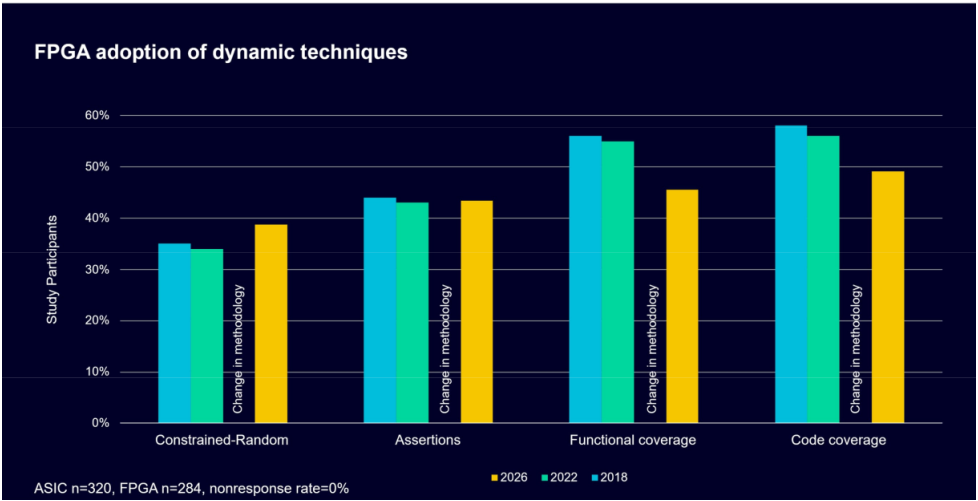

98%
One or more processor cores
82%
Two or more processor cores
4%
Eight or more processor cores
n=320, nonresponse rate=5.6%
FPGA Study Projects
40%
30%
20%
10%
0%
Change in methodology
0
1
2
3 - 7
8 or more
Number of Embedded Processors
2018
2022
2026


FPGA verification language adoption (testbench)
60%
50%
40%
30%
20%
10%
0%
Study Participants
Change in methodology
VHDL
Verilog
System C
SystemVerilog
C/C++
Accellera PSS
Python
FPGA Verification Language Adoption
2018
2022
2026
n=284, nonresponse rate=0%


50%
40%
30%
20%
10%
0%
Study Participants
Change in methodology
Change in methodology
Change in methodology
Change in methodology
Change in methodology
Accellera UVM
OSVVM
UVVM
cocotb
None/Other
FPGA Testbench Methodologies
2018
2022
2026
n=284, nonresponse rate=0%


FPGA assertion language adoption
65%
55%
45%
35%
25%
15%
5%
-5%
Study Participants
Change in methodology
Change in methodology
Change in methodology
Change in methodology
Accellera Open Verification Library (OVL)
SystemVerilog Assertions (SVA)
PSL
Other
FPGA Assertion Language Adoption
n=284, nonresponse rate=0%
2018
2022
2026


40%
30%
20%
10%
0%
Study Participants
Change in methodology
Change in methodology
Formal property checking
Automatic formal verification
FPGA Adoption of Formal Techniques
ASIC n=320, FPGA n=284, nonresponse rate=0%
2018
2022
2026


FPGA adoption of dynamic techniques
60%
50%
40%
30%
20%
10%
0%
Study Participants
Change in methodology
Change in methodology
Change in methodology
Change in methodology
Constrained-Random
Assertions
Functional coverage
Code coverage
2026
2022
2018
ASIC n=320, FPGA n=284, nonresponse rate=0%